Android:
Anatomy of Android apps

LEIM – DAM

Android apps & Linux aspects

- Apps can be written using Kotlin, Java, and C++ language
- Code, data and resource files are packed into an APK file
- Android apps are installed and run on a multi-user linux system
  - Each app is assigned to a new linux user
    - Apps only access files that belongs to their own user
- An app in execution is a Linux process running an ART VM
  - Apps are isolated from other apps
  - When an app is not running and one app component is needed the system creates a process for it
  - When the app is not needed or when the system needs memory for other apps the process is terminated
- Apps need explicit user permission to access device resources like: camera, Bluetooth, device's location

Android app philosophy

- An Android app should be a modular application composed by several components
  - These components provide entry, or interaction, points for the user, the system or other apps to interact with the app
    - User interaction – called **Activities**
    - Background execution – called **Services**
    - System notification – called **Broadcast Receivers**
    - Storage – called **Content Providers**
  - These components are activated and destroyed by the system when needed
    - If the system needs memory the app process and all components may be destroyed
    - To call/use one component we must use **Intent**s

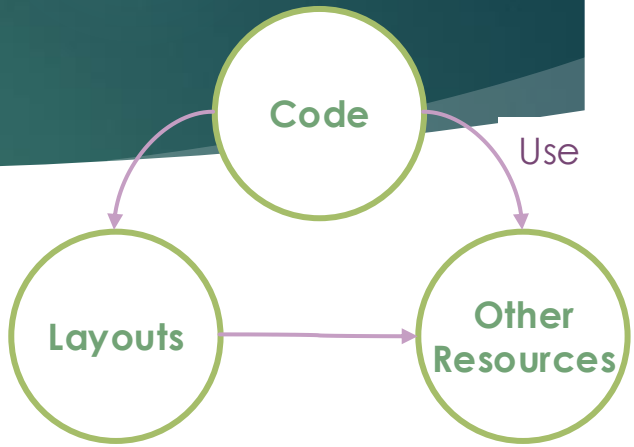
https://developer.android.com/guide/components/fundamentals?hl=en

Android app jumping

- A social network photo sharing scenario:
  - In a social network app we want to take and share a photo – social app triggers camera app
    - Social network app suspended
    - Camera app may triggers other apps, like a File chooser app, which may call other app
      - File Chooser app / camera app suspended
  - The user return to social network app and shares the photo

- Android minimizes the suspended apps, it may destroy them
  - When needed, they will be re-started
  - The modular architecture of Apps should enable this destroy/re-start life-cycle

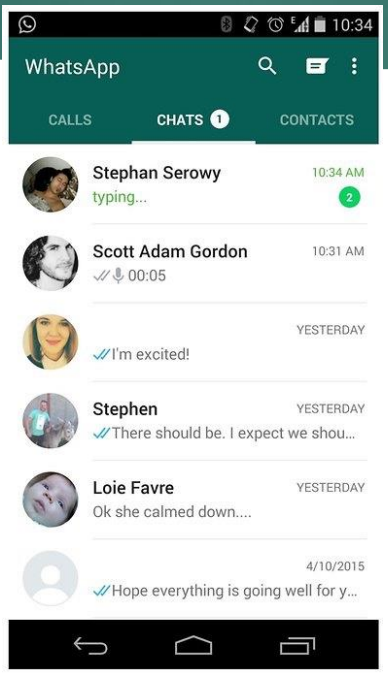
App composition



- An Android app should contain:
  - **Code**
    - The app code in several classes
  - **Resources**
    - **layouts**, images, menus, styles, colors, audio files
    - Being separate from code enable to provide sets of resources to be used depending on device (e.g. screen size) or user settings (e.g. language)
      - enable to adapt user interface without modifying code
  - **AndroidManifest.xml** File
    - Application configuration file where application characteristics are defined
      - Existing app components and their properties (start activity)
      - Interface between the application and the Android operating system
      - User required permissions, hardware and software required
      - Minimum API level required
    - At app installation, the system read this file to evaluates check application requirements

Activities

Introduction



Activity

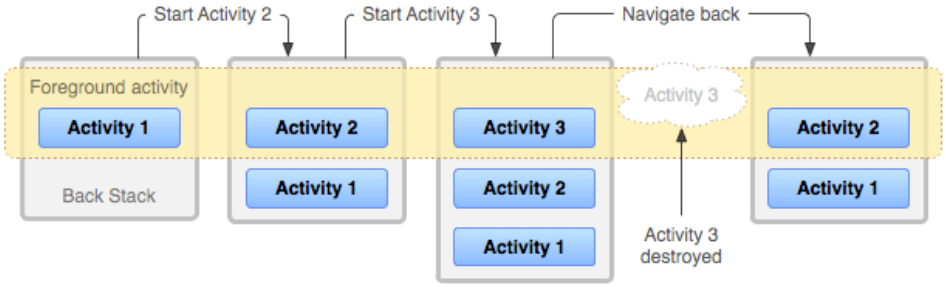
- Provides an app entry point for user interaction
  - One email app might have:
    - one activity that shows a list of new emails,
    - another activity for reading an email, and
    - another activity to compose an email.
- Should contains one screen for one user use case
  - One screen is supported by (one) Layout (ViewGroup) container
    - Layouts contain widgets (Views) to model the interface for user interaction
      - E.g.: TextViews, ImageViews, buttons, …
- The start Activity is just one of them
  - It is specified in the Android Manifest file

Activities: calling other activities

- Activities may call another activities
  - in the same app or in other apps. Example: Email application
    - Activity 1 - shows the list of new emails
    - Activity 2 - reads an email
    - Activity 3 - writes and sends an email
      - Return to Activity 2, then to Activity 1
- The Activity return points are saved in a Stack called **Back Stack**

MainActivity

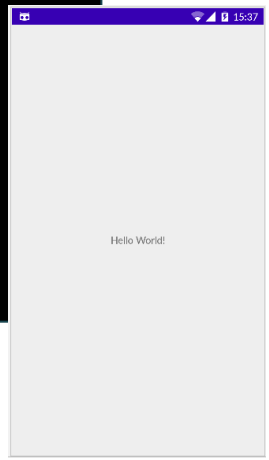
ChatActivity



Activity: “MainActivity.java” file

- An Activity is a subclass of Activity class
  - Subclass of AppCompatActivity to support action bar features

```
class MainActivity : AppCompatActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState) // Call parent class's onCreate method


        setContentView(R.layout.activity_main) // Set layout file for this activity

        …
    }
}
```

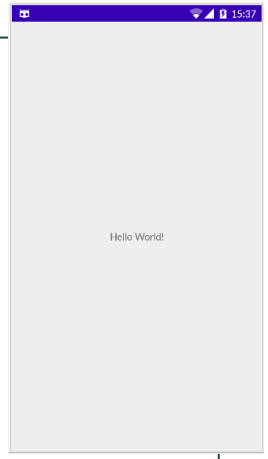


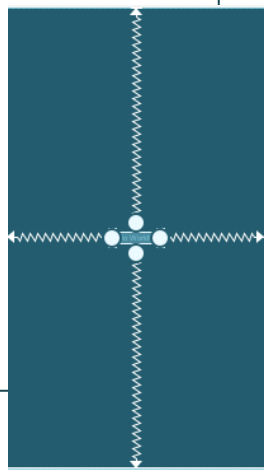
- Activities must be declared in AndroidManifest.xml file
  - Must have an intent-filter to define how they are called
    - Action.MAIN: activity as main entry point and does not receive data
    - Category.LAUNCHER: activity should be displayed in the top-level launcher.

Layout: “activity_main.xml” file

Layout with a TextView with “Hello World!”

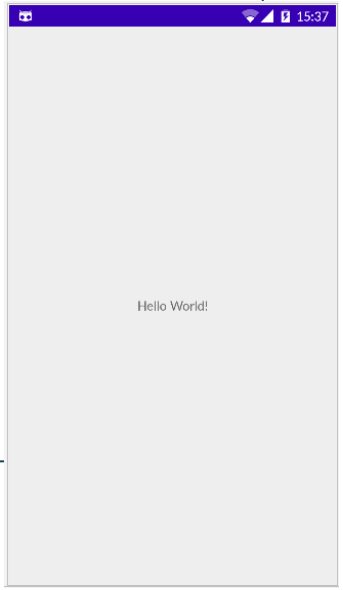
```
<? xml version="1.0" encoding="utf-8"?>
<androidx.constraintlayout.widget.ConstraintLayout
    xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    xmlns:tools="http://schemas.android.com/tools"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    tools:context=".MainActivity">

    <TextView
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Hello World!"
        app:layout_constraintBottom_toBottomOf="parent"
        app:layout_constraintLeft_toLeftOf="parent"
        app:layout_constraintRight_toRightOf="parent"
        app:layout_constraintTop_toTopOf="parent" />

</androidx.constraintlayout.widget.ConstraintLayout>
```





Activity: “AndroidManifest.xml” file

```
<? xml version="1.0" encoding="utf-8"?>
<manifest xmlns:android="http://schemas.android.com/apk/res/android">

   <application
      android:icon="@mipmap/ic_launcher"
      android:label="@string/app_name“
      android:theme="@style/AppTheme">

      <activity android:name=".MainActivity"
         <intent-filter>
            <action android:name="android.intent.action.MAIN" />
            <category android:name="android.intent.category.LAUNCHER" /
         </intent-filter>
      </activity>
   </application>

</manifest>
```



AndroidManifest

- **Configuration**
  - Package name, “target" version, version name, ...
- **Components**
  - Activities, Intents, Services, ...,
- **Permissions**
  - Internet access, access to contact list, ...
- **Libraries**
  - Additional libraries that need to be linked
- **Hardware and Software Features**
  - Video camera, bluetooth or multi-touch screens

AndroidManifest Elements

Element	Description
<activity>	To declare an **activity**
<application>	To declare the application. Contains all elements of the application
<meta-data>	It allows you to associate metadata to an application component
<permission>	Declare security permissions that can be used to limit access
<provider>	To declare a **Content Provider**
<supports-screens>	Let you specify the dimensions of the screens that the application supports
<uses-configuration>	Indicates the hardware and software required in the application
<uses-feature>	Declare a hardware or software feature used in the application
<uses-library>	Declare a shared library that must be "linked" to the application
<uses-permission>	Sets the permissions required for the application to run correctly
<uses-sdk>	It allows to define the minimum, maximum version and the version "target"
…	

Services

Introduction

Services

- Services are one type of app components that can performs (long) background operations/work for the app or other apps
  - Examples: play a background music; download some resource; perform some kind of computation
- They do not provide an user interface
- Services executes in background
  - Activities are "frozen" when the user moves to another Activity
  - Started services tell the system to keep them running until their work is completed
- Activities can start services and let them run or bind to them in order to interact with them.

Services: Declaration and Creation

- A service is a subclass of Service class
  - Needs to override the method onBind
  - Returns the communication channel to the service (Object IBinder?)

```
class MyService : Service() {
   override fun onBind(intent: Intent): IBinder? {
      . . .
      return . . .
   }
}
```

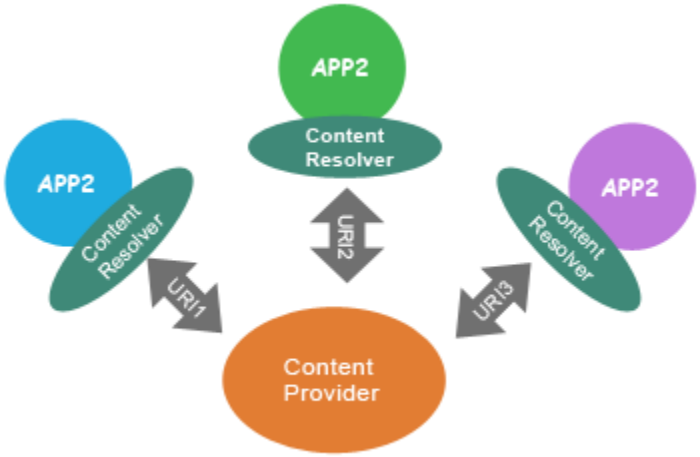
- Services must be declared in AndroidManifest.xml file
  - Apps may use local or remote (from another apps) services

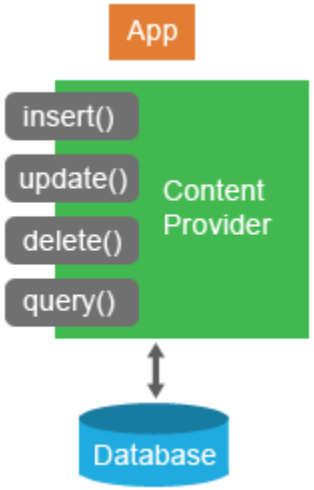
```
<application> ...
   <service
      android:name=".MyService"
      android:enabled="true"
      android:exported="false" >
   </service>
</application>
```

In android:name insert the name of the Service Class. Android Studio gives help, just write <service

Content providers

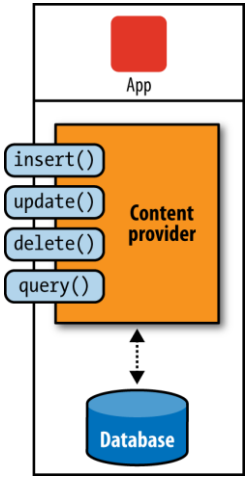
Introduction

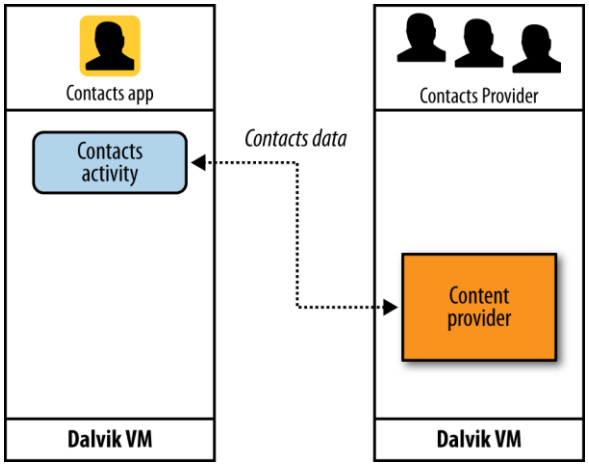




Content Providers

- Used to save and access data within an application or to share data with other applications
  - Through the content provider, other apps can query or modify the data if the content provider allows it
- Data can be in the file system, in a SQLite database, on the Web, or in any other location that the application can access





SQLite databases are databases in Android

Content Providers: Creation

- A ContentProvider is a subclass of ContentProvider class
  - All methods, except onCreate(), may be called by a client application that wants to access the content provide

```
class BookProvider : ContentProvider() {
   override fun onCreate(): Boolean {…}
   override fun query(…): Cursor? {…}
   override fun insert(…): Uri? {…}
   override fun update(…): Int {…}
   override fun delete(…): Int {…}
   override fun getType(…): String? {…}
}
```

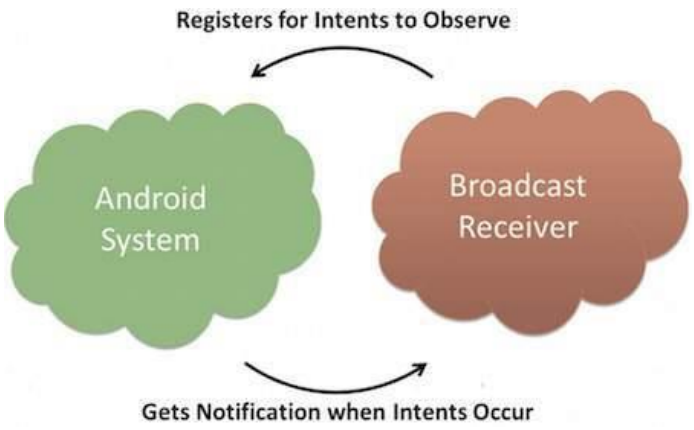
- Declared AndroidManifest.xml file

```
<application> ...
   <provider
      android:name=".BookProvider"
      android:authorities="dam.a6000.bookprovider"
      android:enabled="true"
      android:exported="“true">
   </provider>
</application>
```

The unique identifier of the Content Provider by convention, should start with the package name

Broadcast receivers

Introduction



Broadcast Receivers

- They are app components that enable the reception of system events
  - Like: Low Battery, power connected, system shutdown, alarm, picture captured, ...
  - The system can deliver broadcasts even to apps that aren't currently running
  - They do not provide a user interface but allow to create a status bar with notifications
  - They should perform short actions, sending long action to asynchronous Tasks (using goAsync() or a JobService/JobScheduler)
- Apps can also initiate broadcasts
  - Enabling communication inside and between apps

https://developer.android.com/guide/components/broadcasts.html

Broadcast Receivers: Sending and Receiving

- Sending broadcasts

```
val intent = Intent()
intent.action = "dam.a60000.bookreceiver.action.BOOK_NEW"
intent.addCategory(Intent.CATEGORY_DEFAULT)
intent.data = Uri.parse("http://www.dam.com/9781430248279")
sendBroadcast(intent)
```

- Receiving broadcasts

```
class BookBroadcastReceiver : BroadcastReceiver() {
    override fun onReceive(context: Context, intent: Intent) {
        . . .
    }
}
```

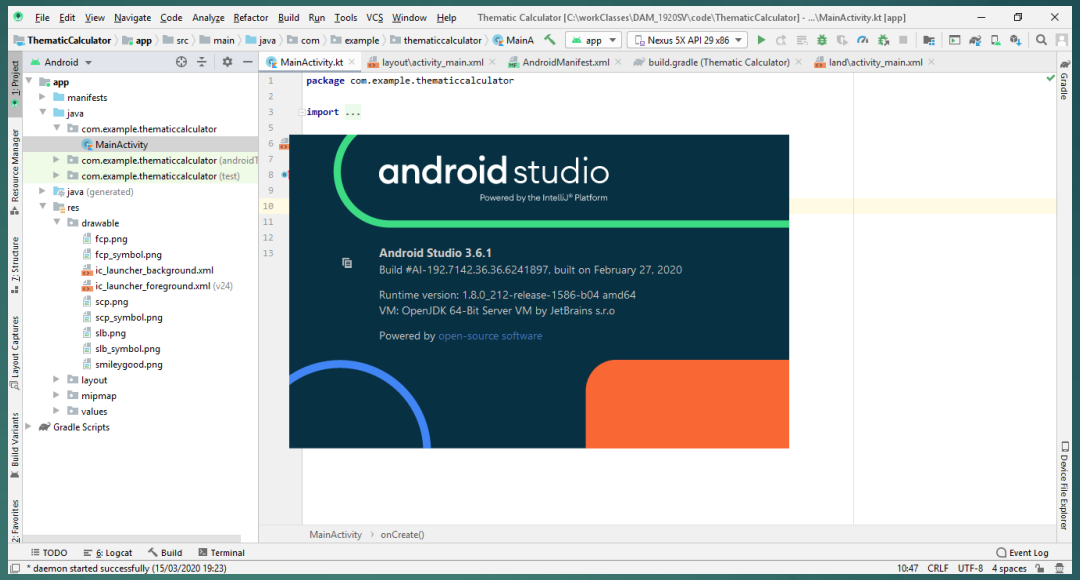
- Communication inside the app

```
LocalBroadcastManager.getInstance(this).sendBroadcast(intent)
// instead of sendBroadcast()
```

Broadcast Receivers: Registration

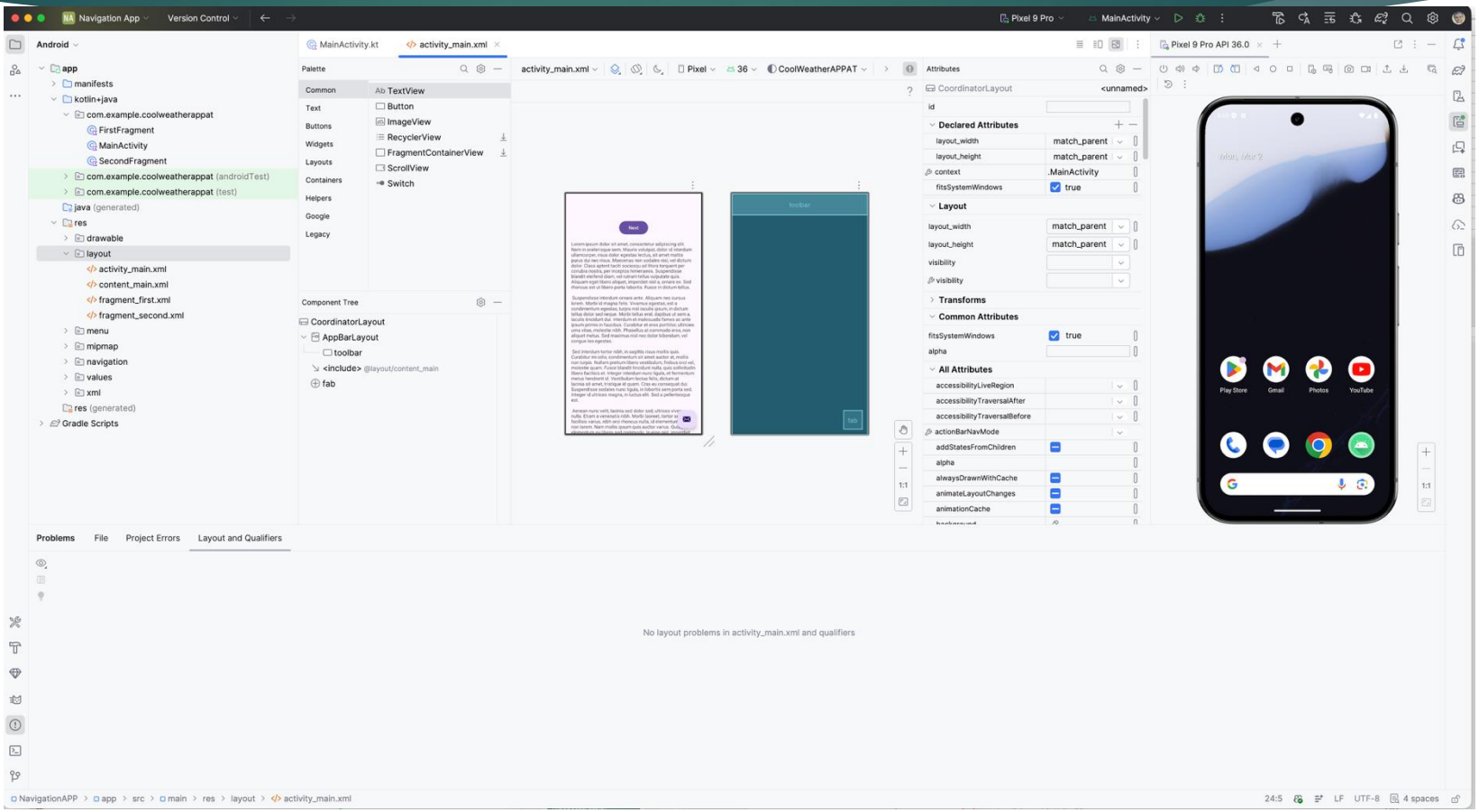
- In AndroidManifest.xml file
  - Apps can register to receive specific broadcasts
    - In Manifest-declared receivers, or
    - In Context-registered receivers (Java Code)

```
<manifest xmlns:android="http://schemas.android.com/apk/res/android”
   package="com.apress.bookprovider">
   <application>
      …
      <receiver
         android:name=".BookBroadcastReceiver"
         android:enabled="true"
         android:exported="true">
         <intent-filter>
            <action android:name="com.ami.bookreceiver.action.BOOK_NEW" />
            <category android:name="android.intent.category.DEFAULT" />
         </intent-filter>
      </receiver>
      …
   </application>
</manifest>
```



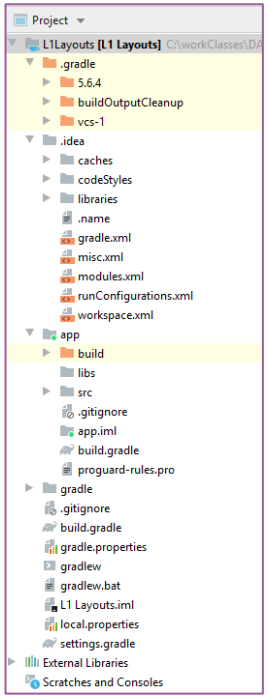
Android Studio

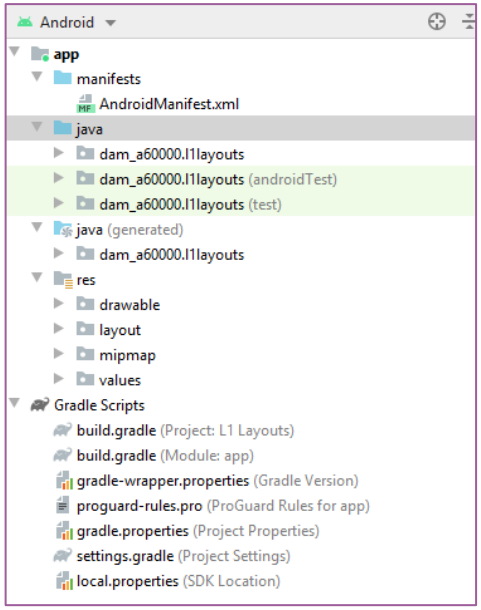
Android Studio

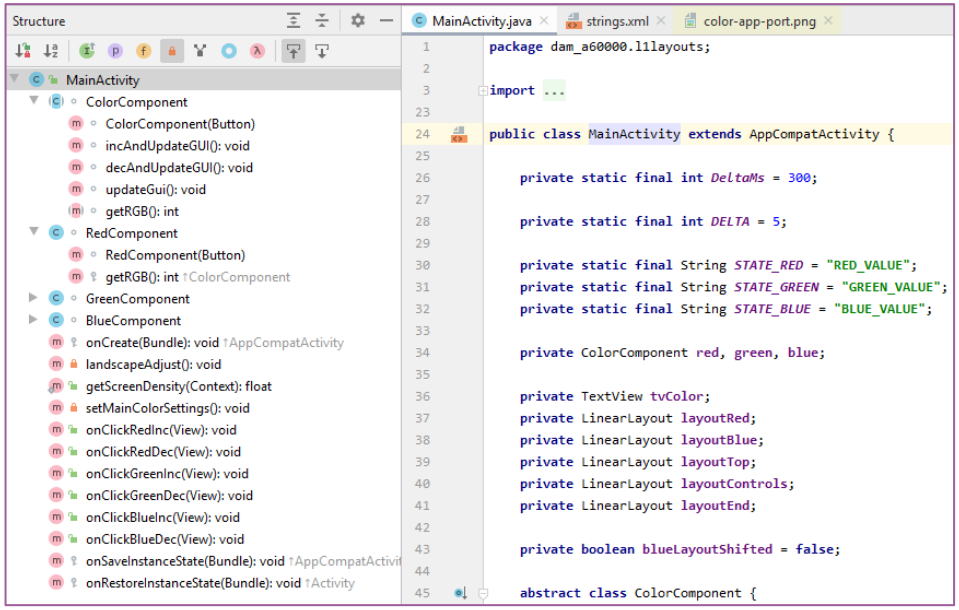


Project and Structure Tool Windows

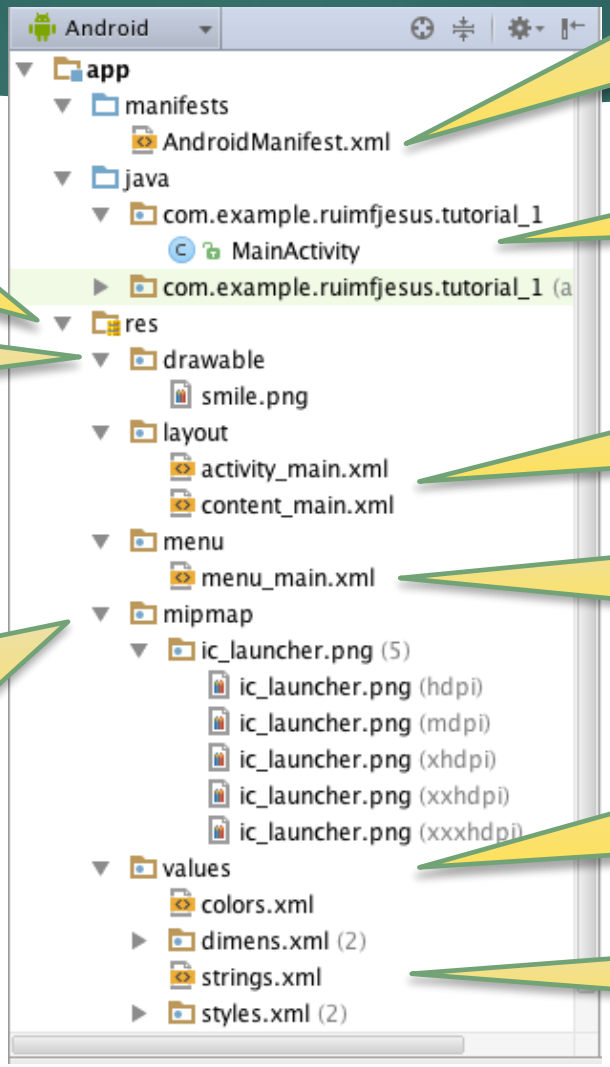
- Project tool window: show project files
  - Project view – show all Project files
  - Android View – Show project files for Android development
- Structure tool window: show the structure of what is seen in editor
  - similar to Outline in Eclipse







Project Elements



Application **configuration** file

Java code of the Activity

All **resources** that are not code

Images used in the application at various resolutions

Activity **Layout**

Application **Menu**

Icons used in various resolutions

Constant data values (e.g., colors, strings, or styles)

Strings used in the application

Use of XML for definitions

- Of application **resources**"
  - **Layouts**, **Menus**, **Views** (User Interface)
  - Values (e.g., **strings.xml**)
- Of application **properties**
  - **AndroidManifest.xml**
- User interface described in XML
  - Separate the layout description from the code that controls it
  - Easier to adapt User Interface to different platforms
  - Visual Editors automatically create XML (Android Studio)

res/layout: User Interface of the Layouts Editor

File Edit View Navigate Code Analyze Refactor Build Run Tools VCS Window Help

T2Thematiccalculator [C:\workClasses\DAM_1920SV\code\T2Thematiccalculator] - ...\activity_main.xml [app]

T2Thematiccalculator > app > src > main > res > layout > activity_main.xml

activity_main.xml | MainActivity.java

```
101        app:layout_constraintGuide_percent="0.05" />
102
103    <androidx.constraintlayout.widget.Guideline
104        android:id="@+id/guideline2"
105        android:layout_width="wrap_content"
106        android:layout_height="wrap_content"
107        android:orientation="vertical"
108        app:layout_constraintGuide_percent="0.9537713" />
109
110    <TextView
111        android:id="@+id/textViewOper1Text"
112        android:layout_width="0dp"
113        android:layout_height="40dp"
114        android:layout_marginTop="60dp"
115        android:background="@android:color/holo_red_dark"
116        android:text="@string/operand_1"
117        android:textAlignment="textStart"
118        android:textColor="@android:color/black"
119        android:textSize="24sp"
120        app:layout_constraintEnd_toStartOf="@+id/guideline2"
121        app:layout_constraintStart_toStartOf="@+id/guideline"
122        app:layout_constraintTop_toBottomOf="@+id/imageButtonSCP"
123
124    <Button
125        android:id="@+id/buttonAdd"
126        android:layout_width="70dp"
127        android:layout_height="wrap_content"
```

Palette: Common, Text, Buttons, Widgets, Layouts, Containers, Google, Legacy — TextView, Button, ImageView, RecyclerView, <fragment>, ScrollView, Switch

Component Tree: container, textViewTop "@string/title_calculator", imageButtonSCP, imageButtonSLB, imageButtonFCP, textViewOper2Text "@string/operand_2", editTextOper2 "@string/Oper2InitialValue", guideline, guideline2, textViewOper1Text "@string/operand_1", buttonAdd "@string/buttonAddText", buttonSub "@string/buttonSubText"

Attributes: textViewOper1Text TextView

id	textViewOper1Text
Declared Attributes	
layout_width	0dp
layout_height	40dp
layout_constraint...	@+id/guideline2
layout_constraintT...	@+id/imageButtonSCF
layout_constraint...	@+id/guideline
layout_marginTop	60dp
background	@android:color/holo_red
id	textViewOper1Text
text	@string/operand_1
textAlignment	textStart
textColor	@android:color/black
textSize	24sp
Layout	

Constraint Widget

androidx.constraintlayout.widget.ConstraintLayout > TextView

Terminal | Build | 6: Logcat | Profiler | 4: Run | TODO | Event Log

Gradle build finished in 2 s 635 ms (20 minutes ago)

110:5 CRLF UTF-8 4 spaces

Applications resources described in XML files

- Strings should be defined in: res/values/strings.xml
  - Enables to change string content to other contexts (languages)
  - Example:

```
<resources>
   <string name="app_name">Tutorial_1</string>
   <string name="action_settings">Settings</string>
   <string name="msg1">Olá Mundo!</string>
   <string name="msg2">Esta é a minha primeira aplicação Android!!</string>
   <string name="image">smile</string>
</resources>
```

- Other values in "values" follow the same principle
  - colors.xml, dimens.xml, styles.xml
  - Resources (string, integer, array, color, dimension, bool, ID) are inside files with **<resources>** as its root element
    - It doesn't matter the file name – what is relevant is the resource declaration

Resources: location and references

Resource Group	Subdirectory	Reference in Code	Reference in XML
Property Animations	animator	R.animator	@animator/<file>
Tween Animations	anim	R.anim	@anim/<file>
Color State List	color	R.color	@color/<file>
Drawables	drawable	R.drawable	@drawable/<file>
Layouts	layout	R.layout	@layout/<file>
Menu	menu	R.menu	@menu/<file>
Raw	raw	R.raw	
Simple Values	values	R.arrays	@bool/<id>
		R.bool	@color/<id>
		R.color	@dimen/<id>
		R.dimen	@integer/<id>
		R.integer	@string/<id>
		R.string	@style/<id>
		R.style	
XML	xml	R.xml	

Resources in code

- Get reference to a View based on resource id
  - findViewById()
- Get resource contents
  - getResources()
    - And use getString, getInteger, …

```
// ID resource
<TextView
   android:id="@+id/textViewHello“
   ... />

// non-ID resources
<string name="hello">Hello</string>

<integer name="initValue">100</integer>
```

```
// get reference to a ID resource
val tvHello = findViewById<TextView>(R.id.textViewHello)

// get a resource string
val str = resources.getString(R.string.hello)

// get a resource Integer
val initvalue = resources.getInteger(R.integer.initValue)
```

Array Resources - String and Integer arrays

- Arrays of Strings and Integers
  - To define in XML: string-array, integer-array
  - In use in code: getStringArray, getIntegerArray

```xml
<?xml version="1.0" encoding="utf-8"?>
<resources>
    <string-array name="planets_array">
        <item>Mercury</item>
        <item>Venus</item>
        <item>Earth</item>
        <item>Mars</item>
    </string-array>
</resources>
```

To create an array file: over res directory select new -> Android Resource File -> keep all (Resource type **Values**, directory name of **values**) and set File name: **arrays**

```kotlin
// get reference to a ID resource
val planets: Array<String> = resources.getStringArray(R.array.planets_array)
```

Array Resources - untyped arrays

- Arrays of untyped elements, called **TypedArrays**
  - Arrays may contain elements of any type in the same array
  - Elements may reference other resources, even drawables or arrays
  - In define in XML: array
  - In use in code: obtainTypedArray

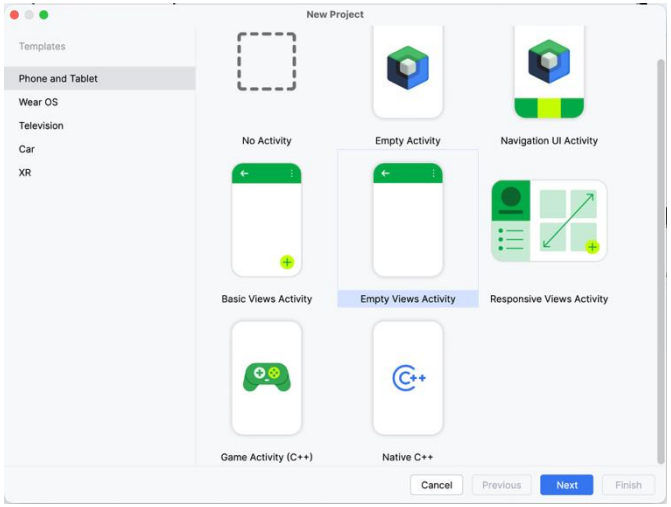
```
<array name="cities">
    <item>@array/lisboa</item>
    <item>@array/porto</item>
</array>

<array name="lisboa">
    <item>Lisboa</item>
    <item>504718</item>
</array>

<array name="porto">
    <item>Porto</item>
    <item>#214349</item>
</array>
```

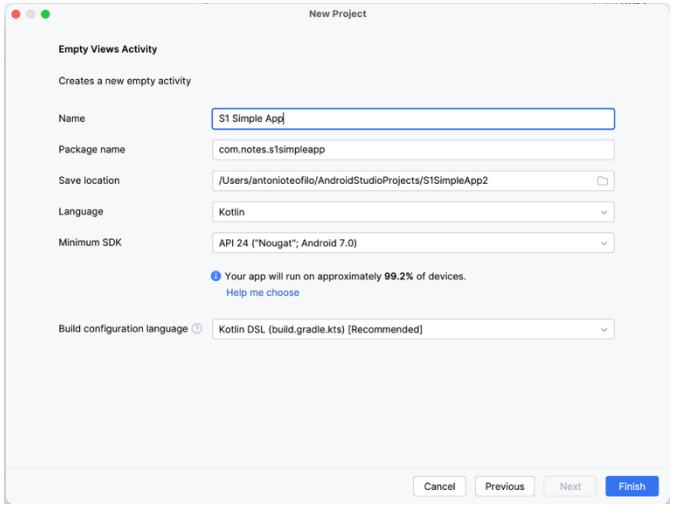
```
val arrayCities = resources.obtainTypedArray(R.array.cities)
for (idx in 0 until arrayCities.length()) {
    val idCity = arrayCities.getResourceId(idx, -1)
    val city = resources.obtainTypedArray(idCity)
    val name = city.getString(0)
    @SuppressLint("ResourceType") val population = city.getInt(1, 0)
    Log.d(LOG_MSG, "City: $name, $population")
}
```

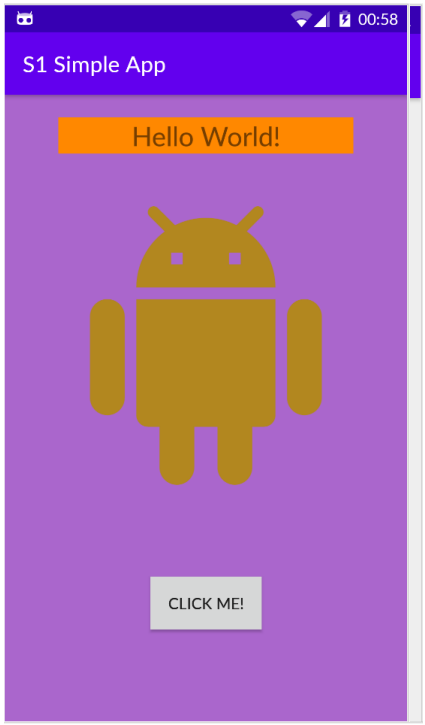
One simple app

A simple app

- Create a simple project with one activity
  - File -> New -> New project -> Empty Activity
    - Name: S1 Simple App, package name: dam_a60000, Finish
- Run the app
  - Next: activity, layout, Strings, manifest, build.gradle

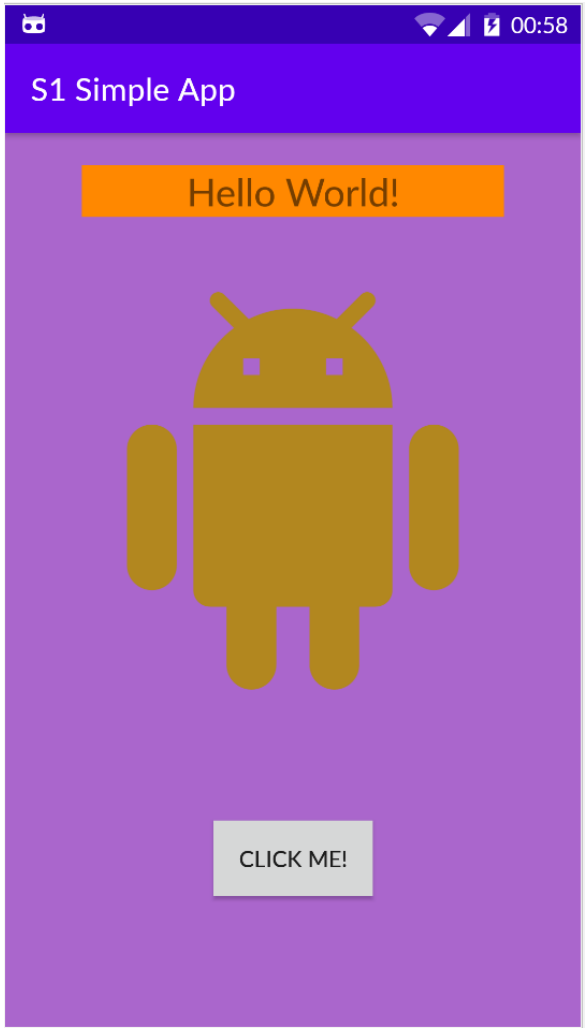






Simple App: layout contents

- Added:
  - TextView: Background color, textSize
  - ImageView: with Android Image Asset
  - Button: with CLICK ME! Text (on Strings.xml)
  - Layout: Background color



Simple app v2: MainActivity

Inherits from AppCompatActivity

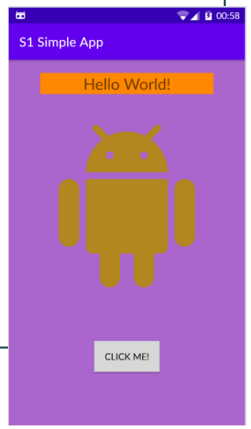
onCreate() called after activity is created. Should initialize it.

Sets activity layout

Sets onClickListener on button

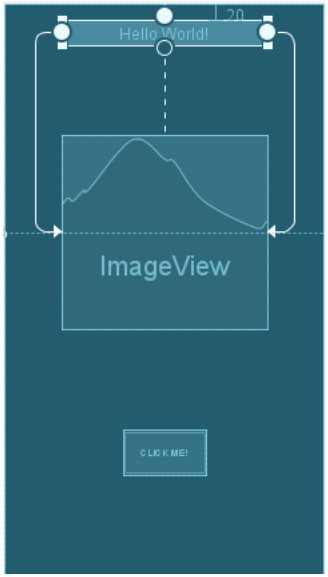
Sets onclick event method to change the layout' background

```
class MainActivity : AppCompatActivity() {
   override fun onCreate(savedInstanceState: Bundle?) {
      super.onCreate(savedInstanceState)
      setContentView(R.layout.activity_main)
      val vgMainLayout = findViewById<ViewGroup>(R.id.MainConstraintLayout)

      val btnClickMe = findViewById<Button>(R.id.buttonClickMe)
      btnClickMe.setOnClickListener {
         var oldColor = Color.GREEN

            val color = (vgMainLayout.background as ColorDrawable).color
            if (color == Color.BLUE)
               vgMainLayout.setBackgroundColor(oldColor)
            else {
               oldColor = color
               vgMainLayout.setBackgroundColor(Color.BLUE)
            }
         }
   }
}
```



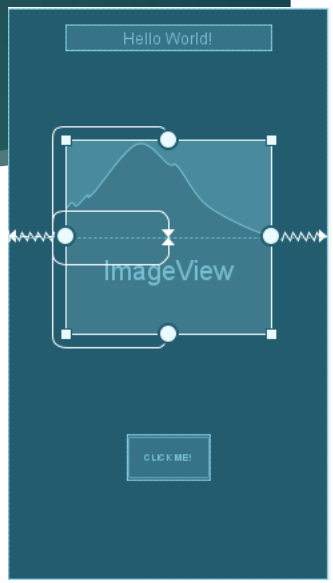
Layout: “activity_main.xml”

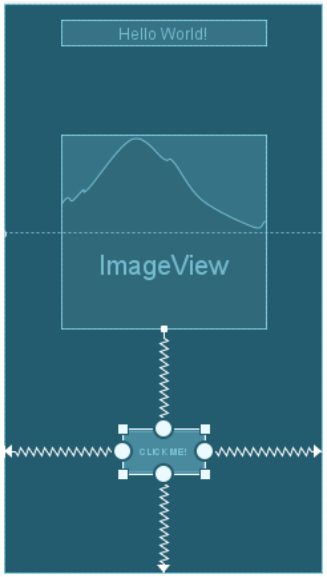
```
<androidx.constraintlayout.widget.ConstraintLayout
    xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    xmlns:tools="http://schemas.android.com/tools"
    android:id="@+id/MainConstraintLayout"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:background="@android:colo/holo_purple"
    tools:context=".MainActivity"
    tools:ignore="ContentDescription">

    <TextView
        android:id="@+id/textViewMain"
        android:layout_width="0dp"
        android:layout_height="wrap_content"
        android:layout_marginTop="20dp"
        android:background="@android:color/holo_orange_dark"
        android:text="@string/hello_str"
        android:textAlignment="center"
        android:textSize="24sp"
        app:layout_constraintEnd_toEndOf="@+id/imageViewAndroid"
        app:layout_constraintStart_toStartOf="@+id/imageViewAndroid"
        app:layout_constraintTop_toTopOf="parent" />
 . . .
```



Layout: “activity_main.xml”

```
. . .
<androidx.constraintlayout.widget.Guideline
    android:id="@+id/guideline"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:orientation="horizontal"
    app:layout_constraintGuide_percent="0.4" />
<ImageView
    android:id="@+id/imageViewAndroid"
    android:layout_width="264dp"
    android:layout_height="249dp"
    app:layout_constraintBottom_toTopOf="@+id/guideline"
    app:layout_constraintEnd_toEndOf="parent"
    app:layout_constraintStart_toStartOf="parent"
    app:layout_constraintTop_toTopOf="@+id/guideline"
    app:srcCompat="@drawable/ic_android_black_24dp" />
<Button
    android:id="@+id/buttonClickMe"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:padding="20dp"
    android:text="@string/button_text"
    app:layout_constraintBottom_toBottomOf="parent"
    app:layout_constraintEnd_toEndOf="parent"
    app:layout_constraintStart_toStartOf="parent"
    app:layout_constraintTop_toBottomOf="@+id/imageViewAndroid" />
</androidx.constraintlayout.widget.ConstraintLayout>
```



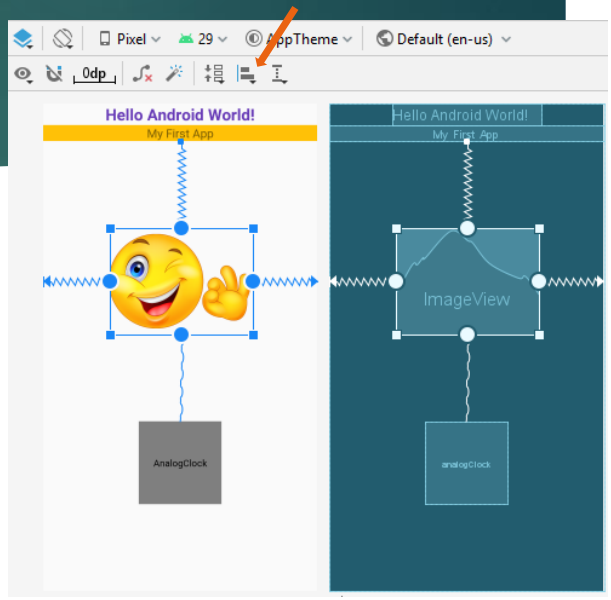


Layout notes

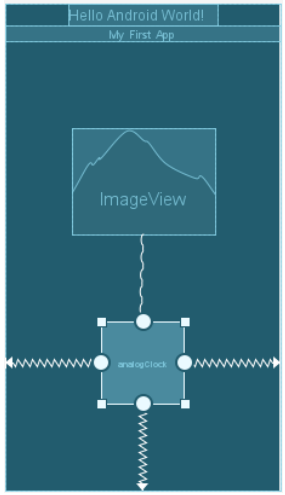
- Used ConstraintLayout
- Used a **HorizontalGuideline** at 50% of vertical size
- **ImageView** with top and bottom constraints connected to Guideline
  - Left and right contraints connected to parent
- **TextView** at top
  - with top constraint to top of layout but with a margin_top
  - Left and right horizontal constraints to left and right size of ImageView
  - With layout_width = 0dp
- **Button**
  - with some padding and constraints to parent and TextView

https://developer.android.com/training/constraint-layout

ConstraintLayout: distribute elements vertically / horizontally



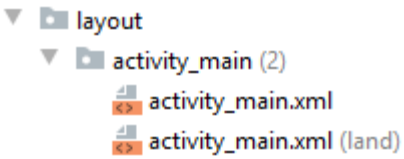
- Distribute elements vertically or horizontally
  - Use double constraints between elements (from one to the other and vice-versa).
  - To set them use Align menu icon and align both elements Vertically or horizontally:
    - vertically in figure (check the constraint visual aspect – is different from normal constraint)
  - Can be used to distribute any number of elements



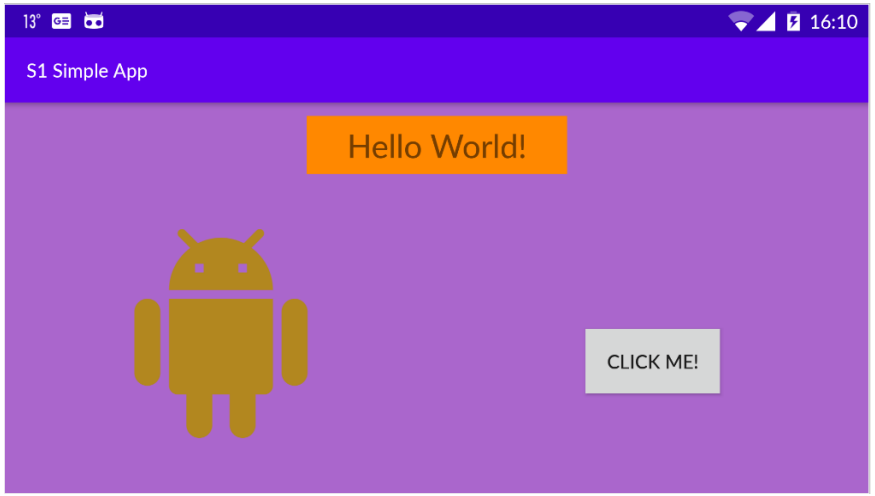
res\values\String.xml

```xml
<resources>
    <string name="app_name">S1 Simple App</string>
    <string name="button_text">Click me!</string>
    <string name="hello_str">Hello World!</string>
</resources>
```

Lanscape variation layout



- Create Lanscape variation
  - In Design editor, in Orientation for Preview icon
  - It is created one more layout with the same name, but with (land)
    - Orgnaize the existing/original componentes in the new landscape variant



AndroidManifest.xml

```xml
<?xml version="1.0" encoding="utf-8"?>
<manifest
   xmlns:android="http://schemas.android.com/apk/res/android"
   package="dam_a60000.s1simpleapp">

   <application
      android:allowBackup="true"
      android:icon="@mipmap/ic_launcher"
      android:label="@string/app_name"
      android:roundIcon="@mipmap/ic_launcher_round"
      android:supportsRtl="true"
      android:theme="@style/AppTheme">

      <activity
         android:name=".MainActivity">
         <intent-filter>
            <action android:name="android.intent.action.MAIN" />
            <category android:name="android.intent.category.LAUNCHER" />
         </intent-filter>
      </activity>

   </application>
</manifest>
```

Package ID: unique name that identifies the package

Application features: icons, name, theme used in formatting, …

Gradle build system

"Gradle is an open-source build automation tool focused on flexibility and performance.

Gradle build scripts are written using a Groovy or Kotlin DSL." *

DSL – Domain Specific Language

* https://docs.gradle.org/current/userguide/userguide.html

Gradle

- Gradle's language plugins establish conventions for discovering and compiling source code.
  - For example, a project applying the Java plugin will automatically compile the code in the directory src/main/java.
  - The last portion of the directory path usually indicates the expected language of the source files.
    - For Kotlin is src/main/kotlin

Groovy | Kotlin

```
.
├── build.gradle
├── settings.gradle
└── src
    └── main
        ├── java
        │   └── HelloWorld.java
        └── kotlin
            └── Utils.kt
```

Groovy | Kotlin

```
.
├── build.gradle.kts
├── settings.gradle.kts
└── src
    └── main
        ├── java
        │   └── HelloWorld.java
        └── kotlin
            └── Utils.kt
```

“build.gradle” file

```
apply plugin: 'com.android.application'
android {
   compileSdkVersion 29
   buildToolsVersion "29.0.2"
   defaultConfig {
      applicationId “dam_a60000.s1simpleapp"
      minSdkVersion 23
      targetSdkVersion 19
      versionCode 1
      versionName "1.0“
      testInstrumentationRunner "androidx.test.runner.AndroidJUnitRunner"
   }
   buildTypes {
      release {
         minifyEnabled false
         proguardFiles getDefaultProguardFile('proguard-android-optimize.txt’),
                                                                  'proguard-rules.pro’
      }
   }
}
dependencies {
   compile fileTree(dir: 'libs’, include:['*.jar’])
   implementation 'androidx.appcompat:appcompat:1.1.0’
   implementation 'androidx.constraintlayout:constraintlayout:1.1.3’
   testImplementation 'junit:junit:4.12’
   androidTestImplementation 'androidx.test.ext:junit:1.1.1’
   androidTestImplementation 'androidx.test.espresso:espresso-core:3.2.0'
}
```

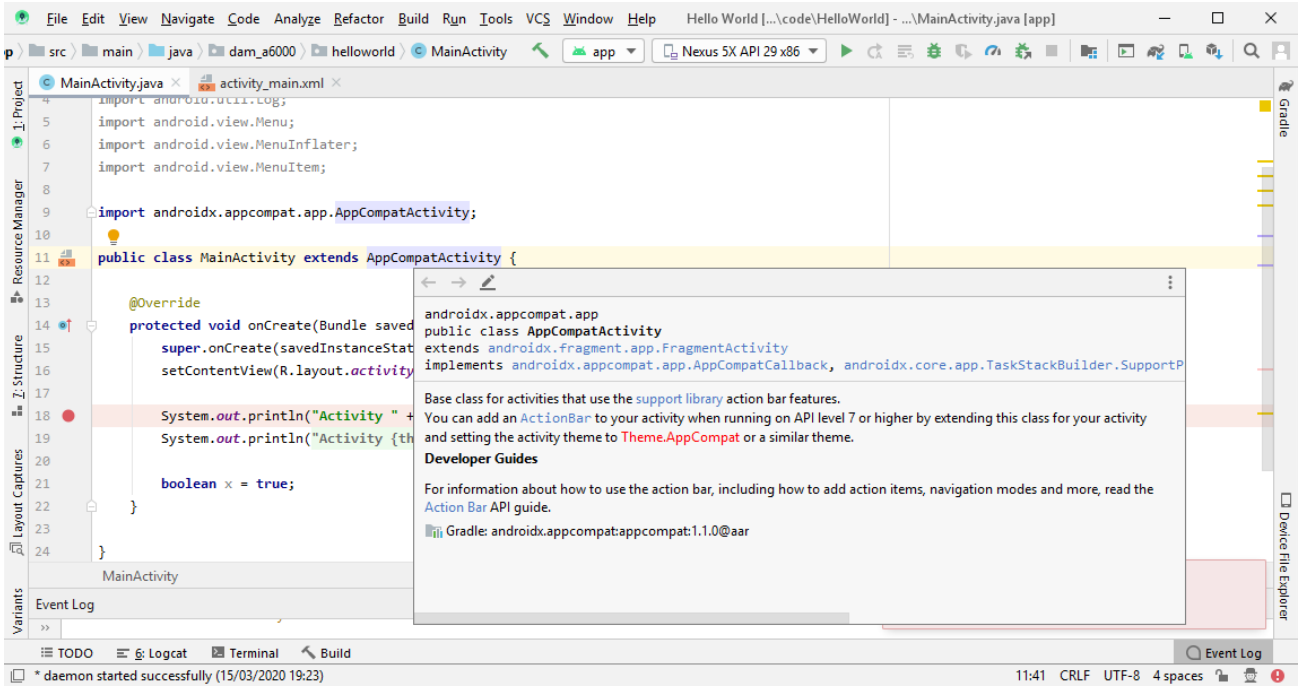
Configuration file for Gradle build system.

Android Studio all files

- app
  - manifests
    - AndroidManifest.xml
  - java
    - dam_a6000.s1simpleapp
      - MainActivity
    - dam_a6000.s1simpleapp (androidTest)
    - dam_a6000.s1simpleapp (test)
  - java (generated)
  - res
- Gradle Scripts
  - build.gradle (Project: S1 Simple App)
  - build.gradle (Module: app)
  - gradle-wrapper.properties (Gradle Version)
  - proguard-rules.pro (ProGuard Rules for app)
  - gradle.properties (Project Properties)
  - settings.gradle (Project Settings)
  - local.properties (SDK Location)

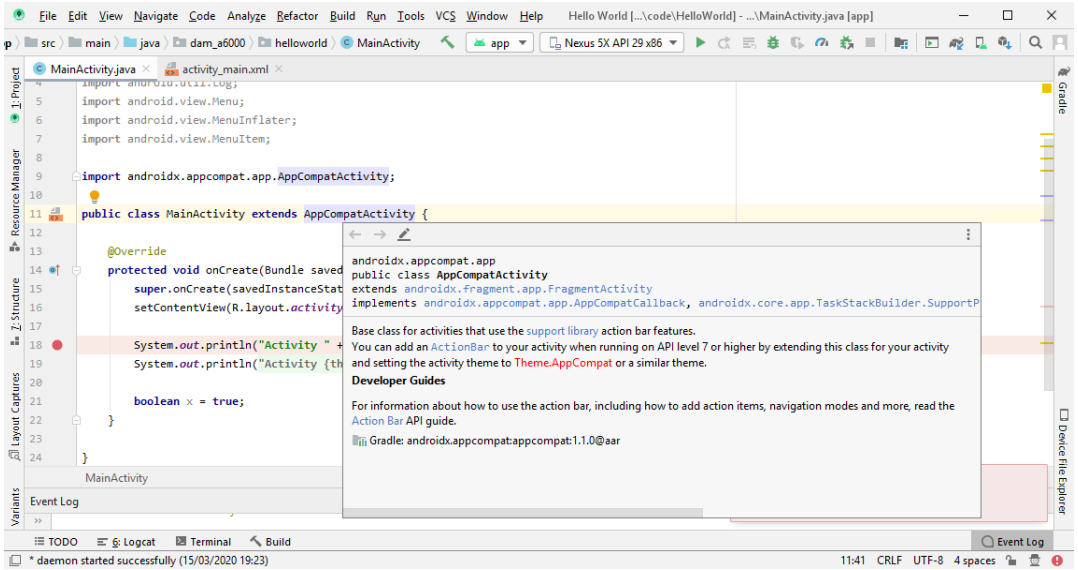
- res
  - drawable
    - ic_android_black_24dp.xml
    - ic_launcher_background.xml
    - ic_launcher_foreground.xml (v24)
  - layout
    - activity_main (2)
      - activity_main.xml
      - activity_main.xml (land)
  - mipmap
    - ic_launcher (6)
      - ic_launcher.png (hdpi)
      - ic_launcher.png (mdpi)
      - ic_launcher.png (xhdpi)
      - ic_launcher.png (xxhdpi)
      - ic_launcher.png (xxxhdpi)
      - ic_launcher.xml (anydpi-v26)
    - ic_launcher_round (6)
  - values
    - colors.xml
    - strings.xml
    - styles.xml

Android Studio: code documentation



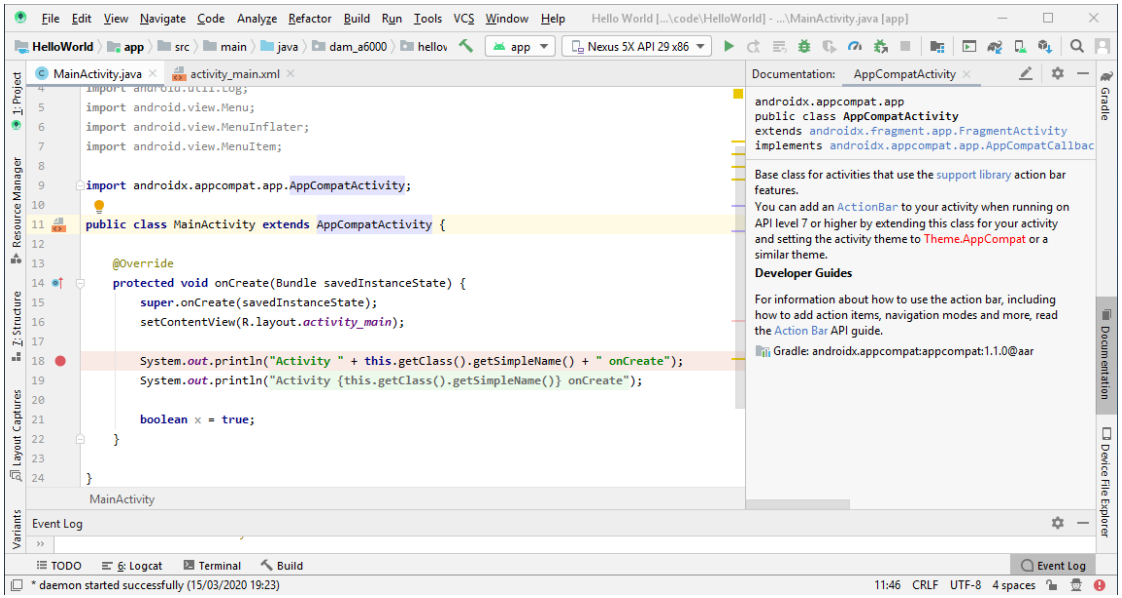
Accessing code documentation

- To open documentation in a **manual pop-up window**: select a "name" and:
  - Press: Ctrl+Q (on PC) or Ctrl+J (on Mac) or
  - Go: Menu -> View -> Quick Documentation
- To open as a Tool Window: open the previous pop-up



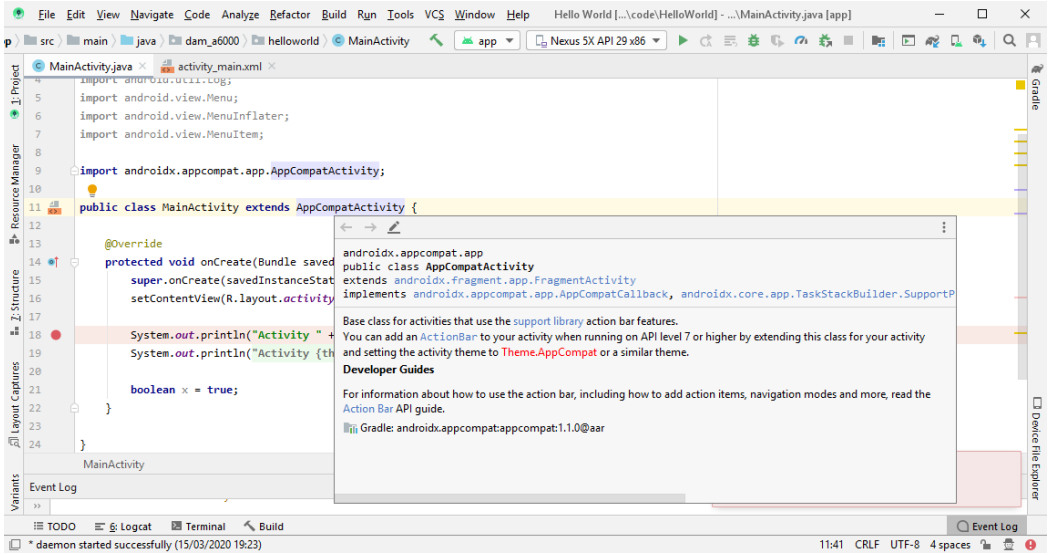
Accessing code documentation

- To open documentation in a permanent **Tool Window**:
  - open the previous pop-up
  - In the top right vertical 3 dots choose: Open as Tool Window
  - You get a tool window that show the current documentation
  - You can minimize it or remove it by choosing Open as Popup in its settings



Accessing code documentation

- To open documentation in an **auto pop-up window**:
  - The documentation window should be in pop-up mode
  - Go: File-> Settings-> Editor-> General: Other: Show quick documentation on mouse over
    - Increase the Delay, if the documentation is annoying you by appearing more often than what you want



Key mapping

- Android Studio have its own key sequences for actions
  - These key sequences are diferente other editors
  - But we can change the key mapping as we want and even we can change the entire mapping for one of the existing configurations
- Change the key mapping to Eclipse key mapping
  - If we want we can use the Eclipse key mapping – so we continue to use some Eclipse familiar key sequences like (but not all are adopted):
    - CTRL+Alt+O to organize imports
    - CTRL+ALT+L to format
  - To change the key mapping to Eclipse: File -> Settings -> keymap -> choose Keymap: Eclipse
- We can always change the key sequence for any action
  - Go to: File -> Settings -> keymap and select the desired action and change its key sequence

Summary

- **Main Components** - Activities, Services, Content Providers, Broadcast Receivers
- **Activities** - a single screen with a user interface
- **Services** - to perform long background operations
- **Content Providers** – data access within and between applications
- **Broadcast Receivers** - to receive and react to broadcast messages from other applications or the system
- **Elements of an Android project** - AndroidManifest.xml file, Java code, resources (drawable, layout, menu, mipmap, values) and build.gradle
- **XML editor and visual user interface** - Android studio contains visual and XML editors to create interface
- **MainActivity class** - activity class extension
- **AndroidManifest file** - configuration file with application features. Interface between the application and the system.
- File **build.gradle** - configuration file that Gradle system uses to build the package. It can modify the Manifest file.
- **Code documentation** – how to acces code documentation in Android Studio

Bibliography

- Application Components
- Resources
- Layouts
- ManifestAndroid
- build.gradle